

Git and Github

Sree Geethika Maduri
Driverless-Taskphase
(of Affiliation)

Formula Manipl
(of Affiliation)

Manipal, India

sree.mitmpl2026@learner.manipal.edu

Abstract— This report covers Git and GitHub concepts commits, branching, merging, conflicts, rebasing, stashing, reflogs, force pushing, and remote push/pull and demonstrated hands-on in a local repository connected to GitHub.

Keywords—Git, GitHub, Version Control, Branching, Rebasing, Reflog, Stash

I. INTRODUCTION

Git is a distributed version control system that tracks changes to files over time, allowing multiple people to work without overwriting each other's changes. GitHub serves as a cloud platform that hosts Git repositories for remote collaboration. This report documents local repository operations, conflict management, and remote repository synchronization using Git Bash.

II. GIT CONCEPTS

A. Commits:

A commit is a snapshot of the project's files at a specific point in time. Git identifies every commit with a unique SHA-1 hash generated from its content, metadata, and a pointer to its parent commit. This chain of parent references forms the project's history, and the full history can be reconstructed by walking backward from any commit.

B. Branching and References

A branch is simply a movable pointer to a specific commit. Creating a new branch lets a developer work on a feature or fix in isolation, without affecting the main line of development, until the work is ready to be merged back in.

C. Merging

Merging combines the changes from one branch into another. If the target branch has not moved since the source branch was created, Git performs a fast-forward merge, simply moving the pointer forward. If both branches have diverged, Git creates a merge commit with two parent commits, joining the two histories together.

D. Merging Conflicts

A merge conflict occurs when the same line(s) of the same file have been changed differently on two branches being merged. Git cannot automatically decide which version to keep, so it pauses the merge and marks the conflicting sections using <<<<<<<<, =====, and >>>>>>>> markers. The developer must manually edit the file to resolve the conflict, then stage and commit the resolved file to complete the merge.

E. Rebasing:

Rebasing takes the commits from one branch and replays them on top of another branch's latest commit, producing a linear history instead of a branching one. Unlike a merge, a rebase rewrites commit hashes, since each replayed commit is technically a new commit with a new parent.

F. Logs and Reference Logs (Reflog)

The 'git log' command shows the commit history reachable from the current branch. The 'git reflog' command records every point HEAD has pointed to on the local machine, including commits no longer part of any branch after operations such as a reset or rebase, making it a safety net for recovering commits that would otherwise appear lost.

G. Stashing

'git stash' temporarily saves uncommitted changes and restores a clean working directory, which is useful when switching branches or contexts without committing unfinished work. The saved changes can later be restored with 'git stash pop'.

H. Force Pushing

A force push overwrites the commit history on the remote repository with the local branch's history. It is typically required after an operation that rewrites history, such as 'git commit --amend' or a rebase. Force pushing is risky in shared repositories, since collaborators who already pulled the old history can end up with conflicting, divergent copies.

I. Remote Repositories: Push and Pull

A remote is a version of the repository hosted elsewhere, such as on GitHub. 'git remote add origin <url>' links a local repository to a remote one. 'git push' uploads local commits to the remote, while 'git pull' downloads and merges commits made on the remote into the local repository.

III. IMPLEMENTATION AND DEMONSTRATION

A local repository was created and used to demonstrate the concepts above directly from the command line. It was initialized with 'git init', and files were staged with 'git add' and recorded with 'git commit', with 'git status' and 'git log' used throughout to check progress.

A. Branching, Merging, and Rebasing

A new branch was created and committed to independently, then merged back with 'git merge', producing a fast-forward merge since the main branch had not moved. To demonstrate a real conflict, the same file was changed differently on two branches; Git paused the merge and

marked the conflicting lines with <<<<<<<, =====, and >>>>>>> markers, which were edited manually before committing the resolution. Separately, two diverging branches were combined with 'git rebase' instead of a merge, replaying one branch's commit on top of the other and producing a linear history rather than a branching one. 'git log --oneline --all --graph' was used throughout to visualize these differences.

B. Stashing and Reflog

An uncommitted file was set aside with 'git stash -u' and restored with 'git stash pop', confirming that stash temporarily holds changes rather than discarding them. 'git reflog' was also used to view every position HEAD had moved through in the session, including moves not visible in 'git log' alone, showing how it can help recover work that would otherwise seem lost.

C. Force Push and Remote Operations

A commit was rewritten with 'git commit --amend', illustrating why a force push is needed to share amended or rebased history with a remote, and why doing so is risky if others have already pulled the old version. The local repository was then connected to GitHub with 'git remote add origin <url>' and pushed with 'git push'. A change made directly on GitHub's website was pulled down locally with 'git pull' to confirm syncing works in both directions, and a separate set of existing solution files was added and pushed into the same repository the same way.

Another type of heading is the "component heading", which is used for other components that aren't part of the main text. These are usually your acknowledgments and your references, which you can see examples of below. These headings are not numbered. The correct styling for them can be applied using the "Heading 5" style, which is the same as the "Heading 1" style but without numbering.

IV. CONCLUSION

This exercise provided a practical understanding of Git's internal model and everyday workflow: how commits are chained together, how branches allow isolated work, how merges and rebases integrate that work in different ways, how conflicts are resolved, and how local repositories stay in sync with a remote on GitHub. Working through real errors, such as an empty commit message or an unresolved conflict, reinforced the reasoning behind each command rather than just its syntax.

REFERENCES

- [1] "Learn Git Branching," *learngitbranching.js.org*. <https://learngitbranching.js.org/>.
- [2] "About Git," GitHub Docs. [Online]. <https://docs.github.com/en/get-started/using-git/about-git>.
- [3] C. Schafer, "Git Tutorial for Beginners: Command-Line Fundamentals," YouTube. <https://www.youtube.com/watch?v=HVsySz-h9r4>.